UNITÀ 3 · APPLICAZIONI JAVA PER LA COMUNICAZIONE IN RETE

Verifica Unità 3: soluzioni e griglia

Risposte corrette, punteggi e indicazioni di correzione. Totale 100 punti; voto in decimi = punti ÷ 10, arrotondato al mezzo voto (60 punti = 6).

1. Scelta multipla · 24 punti

Dom.	Risposta corretta	Punti	Note
a	**ICMP Echo Request e Echo Reply**	3	Tipo 8 e tipo 0.
b	**prova una connessione TCP alla porta 7**	3	Dalla Javadoc: ICMP se ottiene il privilegio, altrimenti TCP porta 7.
c	**4 ms**	3	|44−40| + |42−44| + |48−42| = 4 + 2 + 6 = 12; 12 / 3 = 4.
d	**usare un server più vicino**	3	La latenza dipende soprattutto dalla distanza; la banda no.
e	**un Future**	3	
f	**aspetta un client e restituisce un nuovo Socket**	3	Il SYN lo manda il client; l'handshake lo gestisce il sistema operativo.
g	**101 Switching Protocols**	3	
h	**evitare che proxy e cache li scambino per HTTP**	3	Tranello: la maschera NON è cifratura (XOR con chiave nel frame). Più risposte segnate: 0.

2. Vero o falso · 12 punti (1,5 ciascuna)

N.	V/F	Motivo
1	**F**	ICMP è al livello di rete, dentro IP, senza porte.
2	**F**	Significa solo "nessuna risposta entro il timeout": può esserci un firewall.
3	**V**	Gli argomenti vanno direttamente al programma: niente command injection.
4	**F**	Leggi, somma, scrivi: race condition.
5	**F**	Si connette (backlog del sistema operativo) ma non riceve risposte finché il primo non esce.
6	**V**	Fine del flusso. Una riga vuota è "".
7	**V**	text/event-stream, solo dal server al client.
8	**F**	Dopo il 101 si usano frame con 2-14 byte di intestazione.

3. Leggi l'output del ping · 10 punti

a) Inviati 6, ricevuti 5: persi **1/6 ≈ 16,7%** (accettato 17%) · 2 p.

b) Minimo **18 ms**, massimo **28 ms**, media **(18+22+20+28+20)/5 = 108/5 = 21,6 ms** · 1 p. ciascuno. Errore tipico: dividere per 6 (18 ms): media 0 punti.

c) Differenze 4, 2, 8, 8 → somma 22 → **22/4 = 5,5 ms** · 3 p. (2 p. se il procedimento è giusto con un

errore di conto; il pacchetto perso non entra nel calcolo).
d) **La videochiamata**: audio e video in tempo reale devono arrivare a intervalli regolari; il download tollera ritardi variabili, conta il tempo totale · 2 p. (1 p. senza motivazione).

4. Quanto dura la scansione? · 6 punti

a) **200 × 1 s = 200 s** · b) **200/50 = 4 turni → 4 s** · c) **⌈200/30⌉ = 7 turni → 7 s** (non 6,67 s: l'ultimo turno, anche se incompleto, dura un timeout intero; 6,67 s vale 1 punto). 2 p. ciascuno, 1 p. se c'è il risultato senza calcolo.

5. Trova l'errore · 8 punti

a) **Circa 40 s**: `f.get()` subito dopo `submit` aspetta la fine di ogni compito prima di lanciare il successivo, quindi c'è un solo compito alla volta · 4 p. (2 p. per il tempo, 2 p. per la spiegazione).

```
try (ExecutorService pool = Executors.newFixedThreadPool(20)) {
    List<Future<Esito>> futuri = new ArrayList<>();
    for (String h : hosts) futuri.add(pool.submit(() -> controlla(h)));   // prima lancio tutto
    for (Future<Esito> f : futuri) System.out.println(f.get());           // poi raccolgo
}
```

b) 4 p.: lista di Future (1), tutti i submit nel primo ciclo (2), get nel secondo ciclo (1). Durata attesa: ⌈40/20⌉ = 2 s. Accettate soluzioni equivalenti (invokeAll, stream). Se usa virtual thread con isReachable: corretto ma senza beneficio, nessuna penalità.

6. Completa la chat · 10 punti

(1) **newKeySet** · (2) **5000** (accettata qualsiasi porta valida sopra 1023, coerente) · (3) **accept** · (4) **start** (run è errore: eseguirebbe nel thread del main) · (5) **true** · (6) **readLine** · (7) **null** · (8) **startsWith** · (9) **finally** · (10) **println**. 1 p. ciascuno.

7. Il protocollo in azione · 10 punti (2,5 ciascuno)

Evento	Risposta corretta
1	Solo il nuovo client riceve **ERR serve NICK nome** e viene disconnesso; anna, bruno e carla non ricevono nulla.
2	bruno e carla ricevono **MSG anna pizza stasera?**; anna non riceve nulla.
3	Solo bruno riceve **ERR comando sconosciuto**.
4	anna e bruno ricevono **INFO carla esce dalla chat** (grazie al finally, anche senza QUIT).

Per ogni evento: 1 p. per i destinatari giusti, 1,5 p. per la riga esatta. Formulazioni equivalenti dell'INFO accettate se rispettano il formato.

8. Un handshake catturato · 10 punti

a) **Upgrade: websocket** e **Connection: Upgrade** · 1 p. ciascuno.
b) **concatena la chiave con il GUID fisso 258EAFA5-E914-47DA-95CA-C5AB0DC85B11, calcola lo SHA-1 dei byte, codifica i 20 byte in Base64** · 1 p. per passaggio; il GUID non va ricordato a memoria ("una stringa fissa della RFC" basta).
c) **wss://**, perché da una pagina https il browser blocca le connessioni in chiaro (contenuto misto) e i dati devono essere cifrati con TLS · 1 p.; Origin permette al server di **rifiutare connessioni aperte da pagine di altri siti** · 1 p.
d) Due motivi tra: non si fanno richieste quando non ci sono novità; i messaggi arrivano subito (niente attesa fino al prossimo polling); ogni messaggio ha pochi byte di intestazione invece degli header HTTP; una sola connessione TCP invece di tante · 1,5 p. ciascuno.

9. Progetto · 10 punti

Elemento atteso	Punti
A: controlli TCP sulla porta del servizio (o ICMP motivato, con i limiti di isReachable), timeout dichiarato	2
A: parallelismo motivato (pool fisso o virtual thread con socket) e caso peggiore calcolato, es. 60 servizi, timeout 2 s, virtual thread → circa 2 s, ben sotto i 30 s	2
A: aggiornamento della pagina con SSE (solo server → browser) o polling ogni 30 s, con confronto	2
B: WebSocket motivato (bidirezionale, bassa latenza) contro polling o SSE; formato dei messaggi (es. JSON con tipo, autore, testo)	2
B: almeno due misure di sicurezza: wss://, controllo di Origin, autenticazione degli studenti, validazione dei messaggi, textContent contro XSS	2

Valutare la coerenza delle scelte e la qualità delle motivazioni più della scelta in sé: una soluzione diversa ma ben giustificata (es. WebSocket anche per A) prende il punteggio pieno.